BÀI THỰC HÀNH

LAB 04

Cơ bản về Analog Read

Đọc tín hiệu analog với EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Mục lục

1 Giới thiệu . . . 2
1.1 Tổng quan bài lab . . . 2
1.2 Mục tiêu . . . 2
2 Kiến thức nền . . . 2
2.1 Tín hiệu analog và biểu diễn số . . . 2
2.2 Chuyển đổi tương tự - số . . . 2
2.3 Độ phân giải ADC . . . 3
2.4 Quang trở LDR . . . 3
2.5 Mạch chia áp với LDR . . . 3
3 Thiết lập phần cứng . . . 4
3.1 Phần cứng sử dụng . . . 4
3.2 Ánh xạ chân ADC . . . 4
3.3 Kết nối mạch LDR . . . 4
4 EduFramework API . . . 5
4.1 `analogRead()` . . . 5
5 Bài thực hành . . . 5
5.1 Yêu cầu . . . 5
5.2 Chương trình . . . 5
5.3 Kiểm chứng . . . 6
6 Mở rộng . . . 7
6.1 Đọc điện áp với `analogReadMilliVolts()` . . . 7
6.2 Đọc ADC theo phương pháp non-blocking . . . 7
7 Tài liệu tham khảo . . . 8

1 Giới thiệu

1.1 Tổng quan bài lab

Trong hệ thống nhúng, nhiều đại lượng vật lý không chỉ tồn tại dưới hai trạng thái logic như `HIGH` và `LOW` mà thay đổi liên tục theo điều kiện môi trường. Để vi điều khiển có thể xử lý các tín hiệu dạng này, điện áp analog cần được chuyển đổi thành giá trị số thông qua bộ chuyển đổi tương tự - số (Analog-to-Digital Converter - ADC).

Bài lab này được thiết kế để giới thiệu quá trình đọc tín hiệu analog bằng ADC và API `analogRead()` của EduFramework. Một quang trở LDR được mắc trong mạch chia áp để tạo điện áp thay đổi theo điều kiện chiếu sáng. Giá trị ADC thu được được truyền tới Serial Monitor để quan sát bằng kiến thức đã sử dụng ở bài lab trước.

Bài thực hành tập trung vào giá trị ADC thô thay vì chuyển đổi sang đơn vị lux. Qua đó, mối quan hệ giữa tín hiệu analog, quá trình lấy mẫu, độ phân giải ADC và giá trị số có thể được quan sát trực tiếp trên phần cứng.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Phân biệt tín hiệu analog và biểu diễn số được sử dụng trong hệ thống nhúng.
2. Giải thích các khái niệm cơ bản của ADC gồm sampling, quantization và độ phân giải.
3. Mô tả cách quang trở kết hợp với mạch chia áp để tạo tín hiệu điện áp có thể đưa vào ADC.
4. Sử dụng `analogRead()` để đọc giá trị ADC 12-bit thông qua EduFramework.
5. Xây dựng và kiểm chứng ứng dụng giám sát sự thay đổi ánh sáng trên Serial Monitor.

2 Kiến thức nền

2.1 Tín hiệu analog và biểu diễn số

Tín hiệu analog có thể thay đổi liên tục theo thời gian và biên độ. Trong khi đó, hệ thống số xử lý dữ liệu dưới dạng các giá trị rời rạc. Vì vậy, khi một đại lượng vật lý được chuyển thành điện áp analog, vi điều khiển cần một quá trình chuyển đổi để biểu diễn điện áp đó bằng dữ liệu số.

Sampling là quá trình biến tín hiệu liên tục theo thời gian thành một chuỗi các mẫu rời rạc [1] . ADC thực hiện vai trò quan trọng trong quá trình thu nhận dữ liệu này: tín hiệu điện áp tại đầu vào được lấy mẫu và kết quả được biểu diễn bằng một mã số mà chương trình có thể xử lý.

2.2 Chuyển đổi tương tự - số

Quá trình chuyển đổi analog sang digital có thể được mô tả bằng hai khái niệm cơ bản: sampling và quantization.

Sampling xác định giá trị của tín hiệu analog tại những thời điểm rời rạc. Khoảng thời gian giữa hai lần lấy mẫu liên tiếp được gọi là sampling period, thường ký hiệu là T_s. Sampling frequency F_s có quan hệ:

$$F_s = \frac{1}{T_s}$$

Sau khi lấy mẫu, giá trị biên độ của mẫu cần được biểu diễn bằng một số hữu hạn các mức. Quá trình gán giá trị analog vào một mức số có thể biểu diễn được gọi là **quantization**. Do số mức biểu diễn là hữu hạn, giá trị số thu được chỉ là một biểu diễn rời rạc của điện áp đầu vào. Quantization error là sai khác giữa giá trị analog thực tế và giá trị được biểu diễn sau lượng tử hóa. Sai số này thường không vượt quá một nửa bước lượng tử (0,5 LSB – Least Significant Bit) khi sử dụng phương pháp làm tròn đến mức gần nhất. Tăng độ phân giải ADC giúp giảm kích thước của mỗi bước lượng tử [1] .

2.3 Độ phân giải ADC

Độ phân giải xác định số bit được sử dụng để biểu diễn kết quả chuyển đổi. Với ADC có độ phân giải N bit, số mã số có thể biểu diễn là:

$$2^N$$

S32K1xx tích hợp ADC có độ phân giải tới 12 bit [5] . EduFramework hiện cấu hình đường đọc analog mặc định ở chế độ 12-bit [7] . Vì vậy:

$$2^{12} = 4096$$

Kết quả ADC thô có 4096 mã, từ `0` đến `4095` .

Độ phân giải	Số mã	Dải giá trị thô
12-bit	`4096`	`0` đến `4095`

Bảng 4.2.1. Biểu diễn kết quả ADC 12-bit trong EduFramework

Giá trị `0` và `4095` là các mã số ở hai đầu dải chuyển đổi, không phải đơn vị điện áp hoặc đơn vị ánh sáng. Quan hệ giữa một mã ADC và điện áp phụ thuộc vào điện áp tham chiếu của ADC. Trong EduFramework của bài lab này, độ phân giải đã được framework cấu hình cố định ở 12 bit nên ứng dụng không cần thay đổi độ phân giải trước khi đọc.

2.4 Quang trở LDR

Photoresistor, thường được gọi là LDR (Light-Dependent Resistor), là linh kiện có điện trở thay đổi theo lượng ánh sáng chiếu vào [2] . LDR vì vậy có thể được sử dụng làm phần tử nhạy sáng trong các ứng dụng cần phát hiện sự thay đổi tương đối của điều kiện chiếu sáng.

ADC không đo trực tiếp điện trở của LDR. Để biến sự thay đổi điện trở thành một đại lượng ADC có thể đọc, LDR được kết hợp với một điện trở cố định tạo thành mạch chia áp [2] .

Bài lab sử dụng LDR để quan sát xu hướng thay đổi của ánh sáng, không thực hiện hiệu chuẩn theo đơn vị lux. Giá trị ADC phụ thuộc vào đặc tính của LDR, điện trở cố định, nguồn cấp và điều kiện chiếu sáng thực tế.

2.5 Mạch chia áp với LDR

Mạch chia áp của bài thực hành gồm LDR ở phía nguồn và điện trở cố định `10 kΩ` ở phía GND. Điểm giữa hai linh kiện được nối với đầu vào ADC. Với cấu hình này, điện áp tại điểm đo có thể biểu diễn theo quan hệ chia áp:

$$V_{\mathrm{ADC}} = V_{\mathrm{CC}} \times \left(\frac{R_{\mathrm{fixed}}}{R_{\mathrm{LDR}} + R_{\mathrm{fixed}}} \right)$$

Khi điều kiện chiếu sáng làm điện trở LDR giảm, điện áp tại điểm giữa tăng; ADC vì vậy tạo ra giá trị số lớn hơn. Khi LDR bị che tối, xu hướng ngược lại xuất hiện. Quan hệ này đã được kiểm chứng trên mạch sử dụng trong bài thực hành.

Ghi chú

Giá trị ADC phản ánh điện áp tại điểm giữa của mạch chia áp. Nó không phải là phép đo cường độ ánh sáng theo đơn vị lux nếu chưa có quá trình hiệu chuẩn cảm biến.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài lab sử dụng một LDR và điện trở `10 kΩ` để tạo mạch chia áp bên ngoài. Điện áp tại điểm giữa được đưa vào kênh ADC của S32K144.

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
LDR	Quang trở được sử dụng để tạo tín hiệu thay đổi theo ánh sáng.
Điện trở 10 kΩ	Điện trở cố định tạo mạch chia áp với LDR.
Breadboard	Sử dụng để lắp mạch chia áp.
Jumper wires	Kết nối nguồn, GND và tín hiệu ADC giữa board và mạch ngoài.
USB Cable	Kết nối board với máy tính để cấp nguồn, nạp chương trình và sử dụng Serial Monitor.

Bảng 4.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ chân ADC

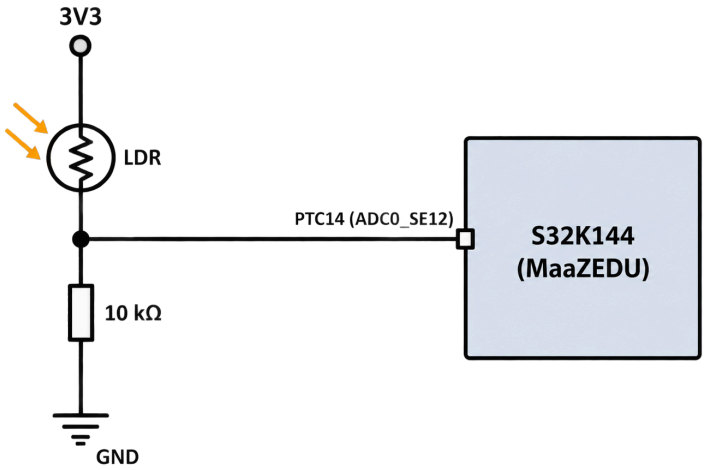
EduFramework hiện hỗ trợ hai Logical Pin cho Analog Input là `ADC0_SE12` và `ADC0_SE13` [7] . Bài thực hành sử dụng `ADC0_SE12` , được ánh xạ tới `PTC14` của S32K144.

Thành phần	Logical Pin	MCU Pin	Chức năng
Ngõ ra mạch chia áp LDR	`ADC0_SE12`	`PTC14`	Analog Input / ADC0 Channel 12

Bảng 4.3.2. Ánh xạ Analog Input sử dụng trong bài thực hành

3.3 Kết nối mạch LDR

LDR được nối từ nguồn của mạch tới điểm đo ADC. Điện trở `10 kΩ` được nối từ điểm đo ADC xuống GND. Điểm giữa được nối tới `ADC0_SE12` (`PTC14`). Board và mạch ngoài phải sử dụng chung GND.



Hình 4.3.1. Sơ đồ kết nối LDR và điện trở 10 kΩ với Analog Input

Với cách mắc đã được kiểm chứng trên phần cứng, tăng ánh sáng làm giá trị ADC tăng; che LDR làm giá trị ADC giảm. Không cần xác định một giá trị ngưỡng cố định trong bài lab vì giá trị thực tế thay đổi theo linh kiện và môi trường.

4 EduFramework API

4.1 `analogRead()`

`analogRead()` thực hiện một lần chuyển đổi ADC trên Logical Pin analog được chỉ định và chờ cho tới khi kết quả sẵn sàng. Trong EduFramework hiện tại, các chân đầu vào được hỗ trợ là `ADC0_SE12` và `ADC0_SE13` [7].

analogRead

Đọc một Analog Input và trả về kết quả ADC thô.

Cú pháp

```
analogRead(pin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
pin	Analog Logical Pin	Chân analog cần đọc. Phiên bản hiện tại hỗ trợ `ADC0_SE12` và `ADC0_SE13`.

Giá trị trả về

Giá trị ADC thô khi chuyển đổi thành công; `-1` nếu chân không hợp lệ hoặc quá trình khởi tạo/chuyển đổi thất bại.

Ví dụ:

```
int value = analogRead(ADC0_SE12);
```

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình đọc LDR qua `ADC0_SE12` và hiển thị liên tục giá trị ADC thô trên Serial Monitor. Chương trình sử dụng baud rate `9600` và cập nhật kết quả sau mỗi khoảng `500 ms`.
Thay đổi điều kiện chiếu sáng bằng cách chiếu sáng vào LDR hoặc che LDR để quan sát xu hướng thay đổi của giá trị ADC.

5.2 Chương trình

Trong `src/main.c`, triển khai chương trình đã được kiểm chứng trên phần cứng như sau:

```
#include "Arduino.h"

int main(void)
{
    uint16_t lightValue = 0U;

    setup();

    Serial1_begin(9600U);

    Serial1_println("EduFramework Analog Read");
    Serial1_println("LDR monitoring started.");

    while (1)
    {
        lightValue = analogRead(ADC0_SE12);

        Serial1_print("ADC: ");
        Serial1_printlnInt(lightValue);

        delay(500U);
    }

    return 0;
}
```

Mã nguồn 4.5.1. Chương trình đọc LDR bằng Analog Input

`setup()` khởi tạo các thành phần nền tảng của EduFramework. Sau đó, `Serial1_begin(9600U)` khởi tạo kênh Serial Monitor theo cấu hình đã sử dụng ở Lab 03.
Trong vòng lặp chính, `analogRead(ADC0_SE12)` thực hiện một lần đọc ADC và lưu kết quả vào `lightValue`. Giá trị này được truyền tới Serial Monitor bằng các API Serial đã học trước đó. `delay(500U)` tạo khoảng cách giữa hai lần hiển thị, giúp kết quả thay đổi đủ chậm để quan sát.
Luồng xử lý của ứng dụng có thể tóm tắt:
`Ánh sáng` → `LDR + mạch chia áp` → `ADC0_SE12` → `analogRead()` → `Serial1` → `Serial Monitor`.

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board, sau đó mở Serial Monitor với baud rate `9600`. Quan sát giá trị ADC ở ba điều kiện: ánh sáng môi trường, chiếu sáng mạnh hơn vào LDR và che LDR. Không cần đạt một giá trị ADC cụ thể; mục tiêu là kiểm tra xu hướng thay đổi ổn định theo điều kiện chiếu sáng.
Dữ liệu trên Serial Monitor có dạng:

```
EduFramework Analog Read
LDR monitoring started.
ADC: ...
ADC: ...
ADC: ...
```

KẾT QUẢ MONG ĐỢI

Serial Monitor cập nhật một giá trị ADC khoảng mỗi `500 ms`. Với mạch chia áp của bài thực hành, giá trị tăng khi LDR được chiếu sáng mạnh hơn và giảm khi LDR bị che tối. Giá trị nằm trong dải mã của ADC 12-bit, từ `0` đến `4095`.

6 Mở rộng

Bài thực hành chính sử dụng `analogRead()` để tập trung vào khái niệm Analog Input và giá trị ADC thô. EduFramework còn cung cấp API chuyển đổi kết quả sang millivolt và nhóm API đọc ADC theo phương pháp non-blocking [7] .

6.1 Đọc điện áp với `analogReadMilliVolts()`

`analogReadMilliVolts()` thực hiện một lần đọc analog và chuyển kết quả ADC thô sang millivolt theo điện áp tham chiếu được cấu hình trong EduFramework.

`analogReadMilliVolts`

Đọc Analog Input và trả về giá trị đã được quy đổi sang millivolt.

Cú pháp

```
analogReadMilliVolts(pin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
`pin`	Analog Logical Pin	Chân analog cần đọc, ví dụ `ADC0_SE12` .

Giá trị trả về

Giá trị điện áp theo millivolt khi thành công; `-1` nếu thao tác thất bại.

Ví dụ:

```
int voltageMv = analogReadMilliVolts(ADC0_SE12);
```

Trong phiên bản EduFramework hiện tại, phép quy đổi của API này sử dụng giá trị tham chiếu mặc định `5000 mV` được cấu hình trong `wiring_analog.c` [7] . Vì vậy, khi sử dụng kết quả như một phép đo điện áp chính xác, điện áp tham chiếu thực tế của hệ thống phải phù hợp với cấu hình được sử dụng trong framework.
Mở rộng: Thay giá trị ADC thô trên Serial Monitor bằng giá trị millivolt và quan sát quan hệ giữa hai cách biểu diễn khi điều kiện chiếu sáng thay đổi.

6.2 Đọc ADC theo phương pháp non-blocking

`analogRead()` là API blocking: hàm bắt đầu chuyển đổi và chỉ trả về sau khi kết quả đã sẵn sàng. Với ứng dụng cần tiếp tục thực hiện công việc khác trong thời gian chờ ADC, EduFramework cung cấp ba API:

- `analogStart(pin)` bắt đầu một lần chuyển đổi và trả về ngay.
- `analogAvailable()` kiểm tra kết quả đã sẵn sàng hay chưa.
- `analogGetResult()` lấy kết quả của lần chuyển đổi đã hoàn thành.

Luồng sử dụng có thể biểu diễn bằng mẫu ngắn sau:

```
analogStart(ADC0_SE12);

/* Other application processing */

if (0U != analogAvailable())
{
    value = analogGetResult();
}
```

Mã nguồn 4.6.1. Nguyên tắc đọc ADC theo phương pháp non-blocking

Mở rộng: Viết lại phần đọc LDR của bài thực hành bằng `analogStart()` → `analogAvailable()` → `analogGetResult()` mà không sử dụng `analogRead()`. Sau khi nhận kết quả, hiển thị giá trị trên Serial Monitor và khởi động lần chuyển đổi tiếp theo. Không thay đổi mạch phần cứng.

7 Tài liệu tham khảo

[1] Iowa State University, *Chapter 6: Sampling Theory* , EE/CprE/HSSE Lab.

[2] Carnegie Mellon University, *Reading a Photoresistor: How Light Is It?* , 60-223 Introduction to Physical Computing.

[3] Arduino, *analogRead()* , Arduino Language Reference.

[4] Arduino, *analogReadResolution()* , Arduino Language Reference.

[5] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet* , S32K1XX, Rev. 15, 2026.

[6] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.

[7] EduFramework, *Analog API* , EduFramework Source Code.